# Practical - 9

## Practical Title: Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

Problem Definition: Implement secure user registration by inserting user details into MySQL using MYSQLi. Validate input on frontend and backend, check duplicate email/username, hash passwords using password_hash(), and display appropriate messages.

## Theory

### 1. Is password_hash() used before storing passwords?

**Answer:** Yes, the PHP password_hash() function is used to create a secure hash of the user's password before storing it in the database. This helps protect passwords and prevents the original password from being stored as plain text.

**Reference Link:**
https://www.w3schools.com/php/func_string_crypt.asp

### 2. Is duplicate email/username checked?

**Answer:** Yes, before inserting a new user, the system checks whether the email address or username already exists in the database. If a duplicate is found, an error message is displayed. Prepared statements are used to execute database queries safely, and a UNIQUE constraint can prevent duplicate values in the database.

**Reference Link:**
https://www.w3schools.com/php/php_mysql_prepared_statements.asp
https://www.w3schools.com/php/php_mysql_prepared_statements.asp
https://www.w3schools.com/mysql/mysql_unique.asp

### 3. Are frontend and backend validations consistent?

**Answer:** Yes, frontend and backend validations should follow the same rules. Frontend validation checks the user's input in the browser, such as empty fields and invalid email formats. Backend validation checks the submitted data on the server before storing it in the database. Backend validation is necessary because frontend validation can be bypassed.

**Reference Link:**
https://www.w3schools.com/php/php_form_validation.asp
https://www.w3schools.com/php/php_form_validation.asp

## A. Implementation/ Output/Result

### registration.php

```
<?php


require_once "db.php";


// Check whether form was submitted using POST
if ($_SERVER["REQUEST_METHOD"] == "POST") {


    // Get form values
    $fullname = trim($_POST["fullname"] ?? "");
    $email = trim($_POST["email"] ?? "");
    $mobile = trim($_POST["mobile"] ?? "");
    $course = trim($_POST["course"] ?? "");
    $year = trim($_POST["year"] ?? "");
    $gender = trim($_POST["gender"] ?? "");
    $password = $_POST["password"] ?? "";
    $confirm_password = $_POST["confirm_password"] ?? "";
    $terms = $_POST["terms"] ?? "";


    // Validation
    $errors = [];
```

```
// Full name
if (!preg_match("/^[A-Za-z\s]{3,}$/", $fullname)) {
    $errors[] = "Please enter a valid full name.";
}

// Email
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors[] = "Please enter a valid email.";
}

// Mobile
if (!preg_match("/^[6-9][0-9]{9}$/", $mobile)) {
    $errors[] = "Please enter a valid 10-digit mobile number.";
}

// Course
if ($course == "") {
    $errors[] = "Please select a course.";
}

// Year
if ($year == "") {
    $errors[] = "Please select your year.";
}

// Gender
```

```
if ($gender == "") {
    $errors[] = "Please select your gender.";
}

// Password
if (!preg_match(
    "/^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[@$!%*?&]).{8,}$/",
    $password
)) {
    $errors[] =
    "Password must contain uppercase, lowercase, number and special character.";
}

// Confirm password
if ($password != $confirm_password) {
    $errors[] = "Passwords do not match.";
}

// Terms
if ($terms != "accepted") {
    $errors[] = "You must accept the Terms and Conditions.";
}

// If there are validation errors
if (!empty($errors)) {
```

```
    echo "<h2>Registration Failed</h2>";
    echo "<ul>";

    foreach ($errors as $error) {
        echo "<li>" . htmlspecialchars($error) . "</li>";
    }

    echo "</ul>";
    echo '<a href="signup.php">Go Back</a>';
    exit;
}

// Hash password before storing it
$hashedPassword = password_hash($password, PASSWORD_DEFAULT);

// Insert into students table
$sql = "INSERT INTO students
        (name, email, password, gender, mobile, course, year)
        VALUES (?, ?, ?, ?, ?, ?, ?)";

$stmt = mysqli_prepare($conn, $sql);

if (!$stmt) {
    die("Database error: " . mysqli_error($conn));
}
```

```
mysqli_stmt_bind_param(
    $stmt,
    "ssssssi",
    $fullname,
    $email,
    $hashedPassword,
    $gender,
    $mobile,
    $course,
    $year
);
// Execute query
if (mysqli_stmt_execute($stmt)) {

    echo "<h2>Registration Successful!</h2>";

    echo "<p>Welcome to StudentHub, "
        . htmlspecialchars($fullname)
        . "!</p>";

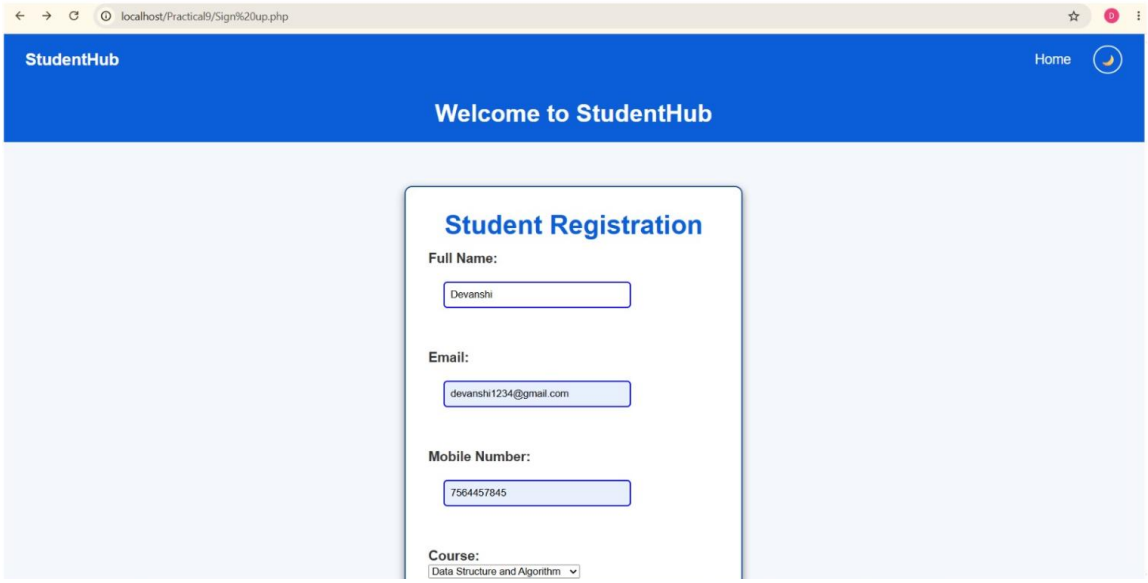
    echo "<p>Your registration has been saved successfully.</p>";
    echo '<a href="signup.php">Go Back to Sign Up</a>';

} else {
    // Duplicate email
    if (mysqli_errno($conn) == 1062) {
```

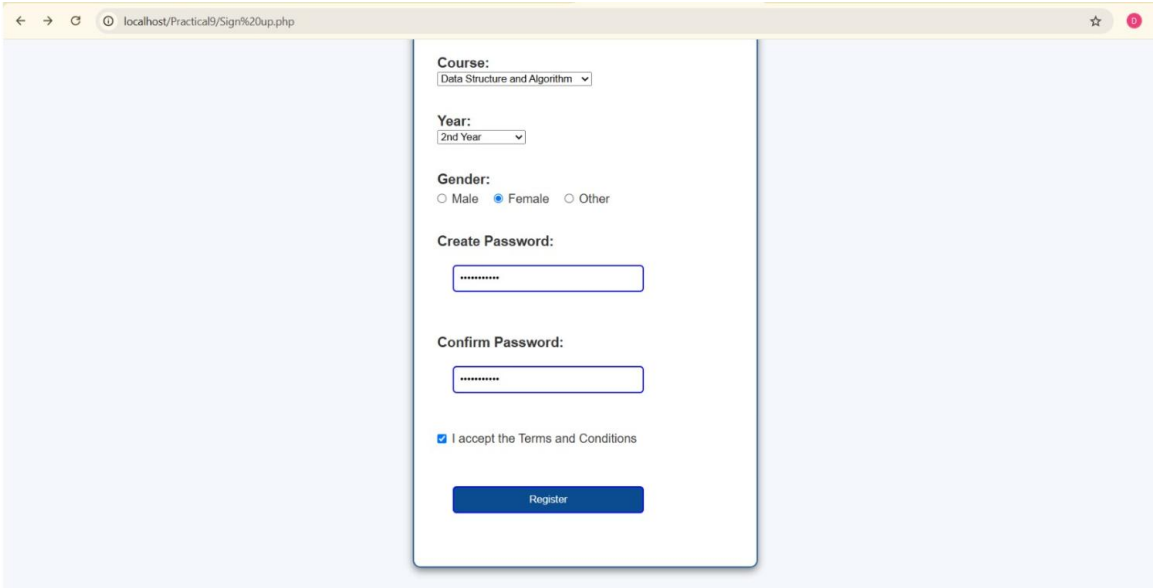
```
            echo "<h2>Email Already Registered</h2>";
            echo "<p>This email address is already registered.</p>";

        } else {
            echo "<h2>Registration Failed</h2>";
            echo "<p>"
                . htmlspecialchars(mysqli_error($conn))
                . "</p>";
        }
    }
    mysqli_stmt_close($stmt);
    mysqli_close($conn);
} else {
    echo "Invalid Request.";
}
?>
```

## B. Evidence of Work (Screenshots)



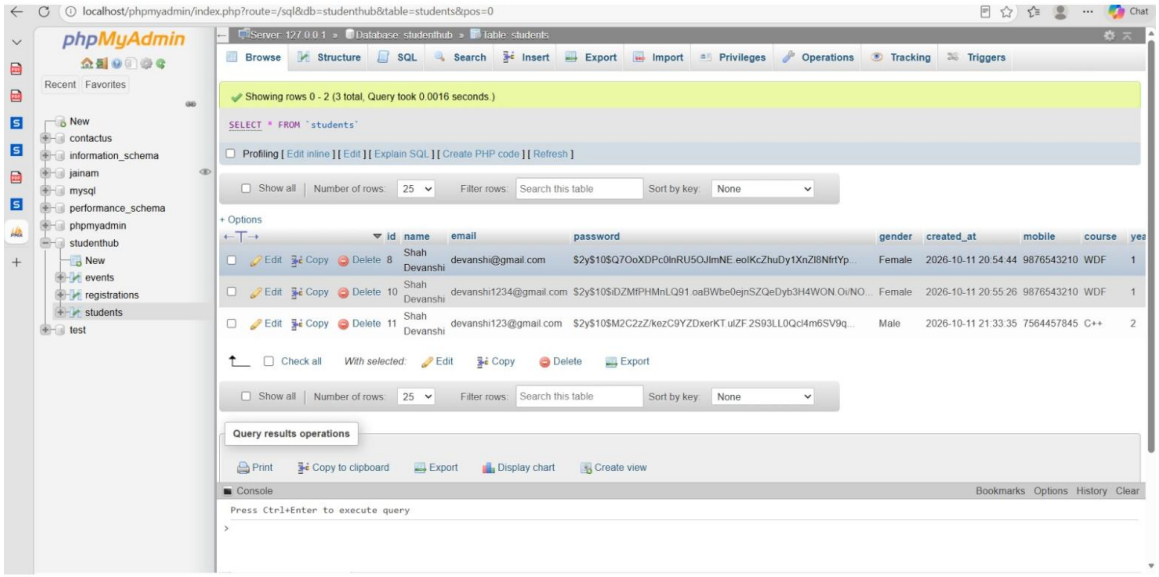

**Description:** The first screenshot shows the StudentHub Student Registration Page opened in a web browser using localhost. The page contains a blue header with the StudentHub logo, a Home link, and a dark mode toggle button. Below the header, the heading “Welcome to StudentHub” is displayed prominently. The registration form is centered on a light-colored background inside a white container with a blue border and shadow. It includes fields for Full Name, Email, Mobile Number, and Course, allowing students to enter their personal details and select their preferred course.



**Description:** The second screenshot shows the lower section of the StudentHub Student Registration Page. It contains dropdown menus for selecting the course and academic year, radio buttons for choosing gender, and input fields for creating and confirming a password. A checkbox is provided to accept the Terms and Conditions, followed by a Register button to submit the form. The sample details are filled in, the Female option is selected, and the checkbox is checked. This section demonstrates the use of different HTML form elements and provides a user-friendly interface for collecting student registration information.




**Description:** The first screenshot shows an error message displayed when a user tries to register with an email address that is already registered in the system. The message "Email Already Registered" informs the user that the email address already exists in the database. This feature prevents duplicate registrations and ensures that each email address is used for a unique student account.



**Description:** The second screenshot shows the students table in the studenthub database using phpMyAdmin. It contains the registered students' information, including student ID, name, email address, hashed password, gender, registration date, mobile number, and course. The existing records help the system check whether an email address has already been registered before allowing a new registration.

## C. GitHub Repository Link:

https://github.com/devanshirshah24-pixel/WDF-Practicals/tree/main/Practical9

## D. Challenges Faced:

- Input validation: Ensuring that all required fields are filled in and contain valid information.
- Duplicate email checking: Preventing registration with an email address or username that already exists.
- Password hashing: Learning how to hash passwords before storing them in the database.
- Database connection: Connecting PHP to MySQL using MySQLi.
- SQL injection prevention: Using prepared statements to handle user input safely.
- Error handling: Displaying appropriate messages for successful registration, invalid input, and duplicate accounts.
- Testing: Testing valid, invalid, and duplicate registration cases to check that the system works correctly.

## E. Learning Outcomes:

- How to create a secure user registration system using PHP and MySQL.
- How to connect PHP with a MySQL database using MySQLi.
- How to validate user input on both the frontend and backend.
- How to check for duplicate email addresses and usernames.

- How to use `password_hash()` to store passwords securely.
- How to use prepared statements to prevent SQL injection.
- How to insert user details into a database.
- How to display appropriate success and error messages.
- How to test registration with valid, invalid, and duplicate inputs.
- The importance of following basic web security practices.

**Faculty Remarks:** (Keep this blank for faculty)